ECommerce API - Postman Documentation

📚 Overview

This Postman collection contains complete API documentation for the ECommerce platform, including all endpoints for:

- **Authentication** (Register, Login, Refresh Token)
- **Products** (CRUD, filtering by brand/category)
- **Product Variants** (Size, Color, Stock management)
- **Shopping Cart** (Add, Remove, Update, Clear)
- **Categories & Brands** (CRUD operations)
- **Addresses** (User shipping addresses management)
- **Orders & Payments** (Create orders, payment intents, coupon management)
- **Coupons** (User coupons, admin assignment, bulk operations)
- **Admin Management** (Cart management, user data, coupon distribution)

🎯 Quick Links

- **Stripe & Orders Testing**: See STRIPE_TESTING_GUIDE.md for complete local payment testing workflow
- **Stripe Setup**: See `../STRIPE_SETUP_GUIDE.md` for configuration

🚀 Getting Started

1. Import the Collection and Environment

1. Open Postman
2. Click **Import** → **Upload Files**
3. Select `ECommerce_API.json` from `/postman/collections/`
4. Select `Development.json` from `/postman/environments/`
5. Set **Development** as the active environment (top-right dropdown)

2. Configure Environment Variables

Edit the **Development** environment with your actual values:

```
baseUrl: http://localhost:7286/api
adminBaseUrl: http://localhost:7286/api/admin
authToken: (Auto-filled after login)
userId: (Auto-filled after login)
```

🔐 Authentication Workflow

1. Register a New User

```
POST /api/auth/register
{
  "userName": "testuser",
  "email": "test@example.com",
  "password": "Test@123"
}
```

Response: `authToken`, `userId`, `refreshToken` (auto-set in environment)

2. Login to Get Token (Test Users Available!)

🧪 **Pre-seeded Test Users:**

Admin:

```
POST /api/auth/login
{
  "email": "admin@ecommerce.com",
  "password": "Admin@123"
}
```

Customers (Pick any):

- `customer@example.com` / `Customer@123` (John Doe - 3 addresses)
- `jane@example.com` / `JaneSmith@123` (Jane Smith - 2 addresses)
- `mike@example.com` / `MikeJohnson@123` (Mike Johnson - 3 addresses)
- `sarah@example.com` / `SarahWilliams@123` (Sarah Williams - 2 addresses)

See TEST_USERS_AND_ADDRESSES.md for complete list with all addresses.

Auto-set variables:

- `authToken` → Used for all authenticated requests
- `userId` → Your user ID
- `refreshToken` → Used to refresh expired tokens

3. Refresh Access Token

```
POST /api/auth/refresh-token
{
  "refreshToken": "{{refreshToken}}"
}
```

Use when: Access token expires (typically after 15-30 minutes)
Returns: New `authToken` (auto-set)

4. Forgot Password (Part 1)

```
POST /api/auth/forgot-password
{
  "email": "test@example.com"
}
```

Effect: Sends OTP code to user's email

5. Verify OTP (Part 2)

```
POST /api/auth/verify-otp
{
  "email": "test@example.com",
  "otp": "123456"
}
```

Effect: Validates OTP code sent to email

6. Reset Password (Part 3)

```
POST /api/auth/reset-password
{
  "email": "test@example.com",
  "otp": "123456",
  "newPassword": "NewPassword@123",
  "confirmPassword": "NewPassword@123"
}
```

Effect: Sets new password if OTP is verified

📦 Product Management

Products WITH Variants (e.g., T-Shirt with sizes)

HasVariants = true

```
POST /api/products/create (Admin)
{
  "name": "Premium T-Shirt",
  "description": "High-quality cotton",
```

```
  "imageUrl": "https://...",
  "price": 29.99,
  "hasVariants": true,
  "stockQuantity": 0,        // Auto-calculated from variants
  "categoryId": 1,
  "brandId": 1
}
```

Then add variants:

```
POST /api/products/{{productId}}/variants
[
  { "size": "S", "color": "Blue", "stockQuantity": 50 },
  { "size": "M", "color": "Blue", "stockQuantity": 60 },
  { "size": "L", "color": "Blue", "stockQuantity": 40 }
]
// Product.StockQuantity = 50 + 60 + 40 = 150 (auto-calculated)
```

Products WITHOUT Variants (e.g., Individual posters)

HasVariants = false

```
POST /api/products/create (Admin)
{
  "name": "Movie Poster",
  "hasVariants": false,
  "stockQuantity": 100,      // Direct product-level stock
  ...
}
```

🛒 Shopping Cart Logic

🔘 Important Business Rules

Rule 1: Products WITH Variants

- Must include `productVariantId`
- Stock checked at **variant level**
- Cannot add without selecting size/color

```
POST /api/cart/items
{
  "quantity": 2,
  "productId": 1,
```

```
  "productVariantId": 5      // ☑ REQUIRED for HasVariants=true
}
```

Rule 2: Products WITHOUT Variants

- Must NOT include `productVariantId`
- Stock checked at **product level**

```
POST /api/cart/items
{
  "quantity": 1,
  "productId": 2
  // ☑ Leave productVariantId empty
}
```

Cart Price Calculation

When adding to cart, price is **snapshotted**:

```
UnitPrice = Product.Price + ProductVariant.AdditionalPrice (if variant)
```

This ensures cart totals don't change if product prices are updated later.

Cart Summary

```
GET /api/cart/summary

Returns:
{
  "cartId": 1,
  "totalItems": 3,
  "subtotal": 100.00,
  "tax": 14.00,              // 14% tax rate
  "discount": 0,             // Future: coupon system
  "total": 114.00
}
```

🖼 Product Images Management (Admin)

Upload Product Image

```
POST /api/products/{{productId}}/images
Content-Type: multipart/form-data

Form Data:
- file: (image file) ← JPG, PNG, GIF, WebP
- isMain: true       ← Set as main product image

Response (200 OK):
{
  "success": true,
  "message": "Image uploaded successfully.",
  "data": {
    "id": 1,
    "productId": 5,
    "imageUrl": "https://res.cloudinary.com/...",
    "isMain": true,
    "createdAt": "2026-03-01T12:30:00Z",
    "updatedAt": "2026-03-01T12:30:00Z"
  }
}
```

Update Product Image

```
PUT /api/products/{{productId}}/images/{{imageId}}
Content-Type: multipart/form-data

Form Data:
- file: (new image file)    ← Replace with new image
- isMain: true              ← Update main status

Response (200 OK):
{
  "success": true,
  "message": "Image updated successfully.",
  "data": {
    "id": 1,
    "productId": 5,
    "imageUrl": "https://res.cloudinary.com/...",
    "isMain": true,
    "createdAt": "2026-03-01T12:30:00Z",
    "updatedAt": "2026-03-01T14:00:00Z"
  }
}
```

Delete Product Image

```
DELETE /api/products/{{productId}}/images/{{imageId}}

Response (200 OK):
{
  "success": true,
  "message": "Image deleted successfully.",
  "data": null
}
```

Note: Old image automatically deleted from Cloudinary

List Product Images

```
GET /api/products/{{productId}}/images?page=1&pageSize=10

Response (200 OK):
{
  "success": true,
  "message": "Images fetched successfully.",
  "data": {
    "items": [
      {
        "id": 1,
        "productId": 5,
        "imageUrl": "https://res.cloudinary.com/...",
        "isMain": true,
        "createdAt": "2026-03-01T12:30:00Z",
        "updatedAt": "2026-03-01T12:30:00Z"
      },
      ...
    ],
    "totalItems": 5,
    "page": 1,
    "pageSize": 10
  }
}
```

Get Single Product Image

```
GET /api/products/{{productId}}/images/{{imageId}}

Response (200 OK):
{
  "success": true,
  "message": "Image fetched successfully.",
  "data": {
    "id": 1,
```

```
    "productId": 5,
    "imageUrl": "https://res.cloudinary.com/...",
    "isMain": true,
    "createdAt": "2026-03-01T12:30:00Z",
    "updatedAt": "2026-03-01T12:30:00Z"
  }
}
```

🖼️ Product Images Notes

- **Admin Only**: All image endpoints require Admin role
- **File Formats**: JPG, JPEG, PNG, GIF, WebP supported
- **Cloudinary Storage**: Images stored on Cloudinary CDN
- **Automatic Deletion**: Old images deleted from Cloudinary when updated/deleted
- **Main Image**: Product.ImageUrl always points to image with `isMain=true`
- **Auto Main Assignment**: If main image deleted, another image automatically promoted to main

---

🔍 Search & Recommendations

Search Products

```
GET /api/products/search?term=shirt&page=1&pageSize=10

Query Parameters:
- term (required)    : Search term (product name/description)
- page (optional)    : Page number (default: 1)
- pageSize (optional): Items per page (default: 10)

Response (200 OK):
{
  "success": true,
  "message": "Products searched",
  "data": {
    "items": [
      {
        "id": 1,
        "name": "Premium T-Shirt",
        "description": "High quality cotton",
        "price": 29.99,
        "categoryId": 1,
        "brandId": 1,
        "hasVariants": true,
        "isFavorite": false,
        "createdAt": "2026-03-01T10:00:00Z"
      },
      {
        "id": 2,
        "name": "Classic Shirt",
```

```
        "description": "Casual wear",
        "price": 24.99,
        "categoryId": 1,
        "brandId": 2,
        "hasVariants": false,
        "isFavorite": true
      }
    ],
    "page": 1,
    "pageSize": 10,
    "totalCount": 2
  }
}
```

Product Search Recommendations (Autocomplete)

```
GET /api/products/search/recommendations?term=shi&size=5

Query Parameters:
- term (required): Prefix to match (min 2 chars)
- size (optional): Max results (default: 5)

Response (200 OK):
{
  "success": true,
  "message": "Recommendations retrieved",
  "data": [
    "Premium T-Shirt",
    "Classic Shirt",
    "Shirt Dress",
    "Shirt Set",
    "Shirt Combo"
  ]
}
```

Search Brands

```
GET /api/brands/search?term=nike&page=1&pageSize=10

Query Parameters: Same as products search

Response (200 OK):
{
  "success": true,
  "message": "Brands searched",
  "data": {
    "items": [
      {
```

```
        "id": 1,
        "name": "Nike",
        "description": "Leading sports brand",
        "createdAt": "2026-03-01T10:00:00Z"
      },
      {
        "id": 5,
        "name": "Nike Pro",
        "description": "Premium Nike line",
        "createdAt": "2026-03-01T10:00:00Z"
      }
    ],
    "page": 1,
    "pageSize": 10,
    "totalCount": 2
  }
}
```

Brand Search Recommendations (Autocomplete)

```
GET /api/brands/search/recommendations?term=nik&size=5

Response (200 OK):
{
  "success": true,
  "message": "Recommendations retrieved",
  "data": ["Nike", "Nike Pro", "Nike Sport", "Nike Premium", "Nike Air"]
}
```

Search Categories

```
GET /api/categories/search?term=clothing&page=1&pageSize=10

Response (200 OK):
{
  "success": true,
  "message": "Categories searched",
  "data": {
    "items": [
      {
        "id": 1,
        "name": "Clothing",
        "description": "All clothing items",
        "parentCategoryId": null,
        "createdAt": "2026-03-01T10:00:00Z"
      },
      {
        "id": 3,
```

```
        "name": "Casual Clothing",
        "description": "Casual wear",
        "parentCategoryId": 1,
        "createdAt": "2026-03-01T10:00:00Z"
      }
    ],
    "page": 1,
    "pageSize": 10,
    "totalCount": 2
  }
}
```

Category Search Recommendations (Autocomplete)

```
GET /api/categories/search/recommendations?term=clot&size=5

Response (200 OK):
{
  "success": true,
  "message": "Recommendations retrieved",
  "data": ["Clothing", "Casual Clothing", "Formal Clothing"]
}
```

🔍 Search Features

- **Full-Text Search**: Uses SQL Server Full-Text Search for performance
- **Graceful Fallback**: If full-text not enabled, falls back to LIKE query
- **Deterministic Results**: Always ordered by Name then ID for consistent pagination
- **Pagination**: All search endpoints support page and pageSize
- **Recommendations**: Returns prefix-matched suggestions for autocomplete UI
- **IsFavorite Flag**: Product search enriches results with user's wishlist status
- **Multi-field Search**: Products search across Name and Description
- **Case-Insensitive**: All searches are case-insensitive

💖 WishList Management

Add Product to WishList

```
POST /api/products/{{productId}}/wishlist

Response (200 OK):
{
  "success": true,
  "message": "Product added to wishlist successfully.",
  "data": {
    "id": 1,
```

```
    "productId": 5,
    "product": {
      "id": 5,
      "name": "Premium T-Shirt",
      "isFavorite": true,
      ...
    },
    "createdAt": "2026-03-01T12:30:00Z"
  }
}
```

Remove Product from WishList

```
DELETE /api/products/{{productId}}/wishlist

Response (200 OK):
{
  "success": true,
  "message": "Product removed from wishlist successfully.",
  "data": null
}
```

Get My WishList

```
GET /api/wishlist?page=1&pageSize=10

Response (200 OK):
{
  "success": true,
  "message": "Wishlist fetched successfully.",
  "data": {
    "items": [
      {
        "id": 1,
        "productId": 5,
        "product": {
          "id": 5,
          "name": "Premium T-Shirt",
          "price": 29.99,
          "isFavorite": true,
          "categoryId": 1,
          "categoryName": "Clothing",
          "brandId": 2,
          "brandName": "Nike",
          ...
        },
        "createdAt": "2026-03-01T10:15:00Z"
      },
```

```
      ...
    ],
    "totalItems": 15,
    "page": 1,
    "pageSize": 10
  }
}
```

Check if Product is in WishList

```
GET /api/products/{{productId}}/wishlist/check

Response (200 OK):
{
  "success": true,
  "message": "Wishlist item check completed.",
  "data": true          // or false
}
```

🏷 WishList Notes

- **Duplicates**: Cannot add same product twice (returns error if already exists)
- **IsFavorite Flag**: Automatically set to `true` for products in wishlist
- **Pagination**: Wishlist supports page and pageSize parameters
- **Auth Required**: All wishlist endpoints require authentication
- **Product Details**: Returns full ProductDto with all details

---

🏢 Admin Endpoints

Cart Management (Admin Only)

```
GET /api/admin/carts                          // List all carts
GET /api/admin/carts/{{cartId}}              // Get specific cart
GET /api/admin/carts/user/{{userId}}         // Get user's cart
GET /api/admin/cart-items/cart/{{cartId}}    // List items in cart
DELETE /api/admin/carts/{{cartId}}           // Delete cart
DELETE /api/admin/cart-items/{{cartId}}      // Delete cart item
```

📊 API Response Format

Success Response (200 OK)

```
{
  "status": true,
  "message": "Product created successfully.",
  "data": {
    "id": 1,
    "name": "T-Shirt",
    ...
  }
}
```

Error Response (400 Bad Request)

```
{
  "status": false,
  "message": "Product not found.",
  "data": null
}
```

Paginated Response

```
{
  "status": true,
  "message": "Products fetched successfully.",
  "data": {
    "items": [...],
    "totalItems": 50,
    "page": 1,
    "pageSize": 10
  }
}
```

🔄 Stock Management Flow

Automatic Stock Calculation (Products WITH Variants)

1. **Create Variant** → Product.HasVariants = true
2. **Update Variant Stock** → Product.StockQuantity auto-recalculated
3. **Delete Last Variant** → Product.HasVariants = false, StockQuantity = 0

Stock Validation

- **Adding to cart**: Check available stock before adding
- **Updating quantity**: Re-validate stock for new quantity
- **Cannot delete variant** if items are in active carts

🔍 Useful Query Parameters

Pagination

```
?page=1&pageSize=10      // Default pagination
```

Filtering

```
GET /api/products/by-category/{{categoryId}}
GET /api/products/by-brand/{{brandId}}
```

📝 Common Workflows

Workflow 1: Create Product with Variants

```
1. POST /api/products/create              → Get productId
2. POST /api/products/{{productId}}/variants (3x)  → Add S, M, L sizes
3. GET /api/products/{{productId}}        → View auto-calculated stock
```

Workflow 2: Customer Shopping

```
1. POST /api/auth/login                   → Get authToken
2. GET /api/products                      → Browse products
3. POST /api/cart/items                   → Add to cart
4. GET /api/cart/summary                  → View total
5. DELETE /api/cart/items/{{id}}          → Remove items
```

Workflow 3: Admin Stock Management

```
1. GET /api/admin/products                → Find product
2. PUT /api/products/{{id}}/variants/{{variantId}}  → Update stock
3. GET /api/admin/carts                   → Check all carts
```

🐛 Troubleshooting

Error: "This product has variants. You must select a variant."

- Solution: For products with `HasVariants=true`, add `productVariantId` to cart request

Error: "This product does not have variants. VariantId must not be provided."

- Solution: For products with `HasVariants=false`, remove `productVariantId` from cart request

Error: "Insufficient stock"

- Check product stock: `GET /api/products/{{productId}}`
- For variants: Check specific variant availability
- Stock includes items already in cart

Error: "Unauthorized" (401)

- Ensure authToken is set in environment
- Re-login to get fresh token
- Check Authorization header is set to Bearer token

📍 Addresses Management

⚠ **Important**: Users must create at least one address before placing orders.

Get My Addresses

```
GET /api/addresses
```

Returns: List of user's saved addresses

Create Address

```
POST /api/addresses
{
  "country": "USA",
  "city": "New York",
  "street": "123 Main St, Apt 4B",
  "postalCode": "10001"
}
```

Response:

```
{
  "success": true,
  "message": "Address created successfully.",
  "data": {
    "id": 1,
```

```
    "country": "USA",
    "city": "New York",
    "street": "123 Main St, Apt 4B",
    "postalCode": "10001"
  }
}
```

Update Address

```
PUT /api/addresses/{id}
{
  "country": "USA",
  "city": "Los Angeles",
  "street": "456 Oak Ave",
  "postalCode": "90001"
}
```

Delete Address

```
DELETE /api/addresses/{id}
```

Note: Cannot delete an address if it's used in active orders.

💳 Orders & Payments

Create an Order

```
POST /api/orders
{
  "addressId": 1,
  "orderItems": [
    {
      "productId": 1,
      "quantity": 2,
      "price": 99.99
    }
  ],
  "couponCode": "SAVE10"
}
```

Response: `{ "id": 101, "status": "Pending", "totalAmount": 189.98 }`

Create Payment Intent for Order

```
POST /api/orders/{{orderId}}/payment
{
  "paymentMethodType": "card"
}
```

Response: `{ "clientSecret": "pi_test_xx", "paymentIntentId": "pi_test_xx" }`

Get Order Details

```
GET /api/orders/{{orderId}}
```

Get User's Orders

```
GET /api/orders/my-orders?page=1&limit=10
```

🎟 Coupons & User Access

Get My Available Coupons (User)

```
GET /api/coupons/my-coupons?page=1&limit=10
```

Only returns coupons assigned to current user

Get Active Coupons (Public)

```
GET /api/coupons/active
```

Assign Coupon to User (Admin)

```
POST /api/coupons/{{couponId}}/assign-user/{{userId}}
{
  "canUse": true
}
```

Bulk Assign Coupon (Admin)

```
POST /api/coupons/{{couponId}}/assign-users
{
  "userIds": ["user-1", "user-2", "user-3"],
  "canUse": true
}
```

Remove Coupon from User (Admin)

```
DELETE /api/coupons/{{couponId}}/remove-user/{{userId}}
```

Get Coupon Users (Admin)

```
GET /api/coupons/{{couponId}}/users?page=1&limit=10
```

Returns: List of users assigned to coupon with their usage info

---

🔔 Testing Payment Flow

⚠️ **Important**: Before testing payments locally, you need:

1. **Stripe CLI running**:

   ```
   stripe listen --forward-to https://localhost:7286/api/webhooks/stripe
   ```

2. **Application running**:

   ```
   dotnet run
   ```

3. **See STRIPE_TESTING_GUIDE.md** for complete step-by-step flow with:
   - Test card numbers
   - Payment confirmation in Stripe Dashboard
   - Webhook verification
   - Coupon application testing

⚙️ 📋 Environment Variables

Variable	Purpose	Example

Variable	Purpose	Example
`baseUrl`	API base URL	`http://localhost:7286/api`
`adminBaseUrl`	Admin endpoints	`http://localhost:7286/api/admin`
`authToken`	JWT token (auto-set)	`Bearer eyJhbGc...`
`userId`	Current user ID (auto-set)	`user-123`
`productId`	Product ID for testing	`1`
`variantId`	Variant ID for testing	`1`
`cartId`	Cart ID for admin testing	`1`
`categoryId`	Category ID	`1`
`brandId`	Brand ID	`1`
`orderId`	Order ID (set after create)	`101`
`addressId`	Address ID for orders	`1`
`couponCode`	Coupon code to test	`SAVE10`
`couponId`	Coupon ID for admin ops	`1`
`page`	Pagination page	`1`
`pageSize`	Items per page	`10`

☑ Best Practices

1. **Always set authToken** in environment after login
2. **Use collection variables** (`{{productId}}`) instead of hardcoding IDs
3. **Test with pagination** for large datasets
4. **Validate stock** before adding to cart in UI
5. **Handle variant selection** for products with multiple options
6. **Cache computed stock** on client for better UX

📞 API Support

For more information, refer to:

- Base URL: `http://localhost:7286`
- Swagger/OpenAPI: `/swagger/index.html` (if available)
- Environment: Development (local testing)

Last Updated: March 4, 2026